

```

"""
test_blob_scout.py
=====
Deux tests rapides, méthodologiquement propres, pour scout :
- TEST A : Sweep dimension ambiante  $d_{\text{amb}} \in \{1, 2, 3, 4, 5\}$ 
    → analogue Mermin-Wagner. K4 forme-t-il une transition autour de  $d=3$ 
- TEST B : Cascade triangle → tétraèdre ( $n_{\text{K4}}$  vs  $n_{\text{tri}}$  en log-log)
    → exposant  $b$  par rapport à baseline Erdős-Rényi ( $b=2$ )

Méthodologie : multi-seeds avec barres d'erreur, banque de seeds fixée,
aucun paramètre ajusté. Données brutes saisies.

Objectif : décider si le scale-up Colab vaut le coup (signal vs bruit).
"""

import numpy as np
from scipy.spatial import distance_matrix
from scipy.special import gamma as gamma_func
import time
import json

# ----- Paramètres V109 (fixés, pas d'ajustement) -----
KAPPA0 = 8
BETA = 0.30
TAU = 0.50

# ----- Paramètres scout (léger pour exécution interne) -----
N = 150          # taille graphe
N_STEPS = 800    # pas MCMC
SAMPLE_EVERY = 40 # un échantillon tous les 40 pas → ~20 par run
SEEDS = [0, 1, 2, 3] # banque fixée
DIMS = [1, 2, 3, 4, 5]

# =====
def rips_radius(N, d, kappa0=KAPPA0):
    """Rayon  $r$  tel que  $N * V_d(r) \approx \text{kappa0}$  (degré moyen calibré)."""
    V = np.pi**((d/2) / gamma_func(d/2 + 1))
    return (kappa0 / (N * V))**(1/d)

def build_rips_dim(N, d, seed):
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0, 1, (N, d))
    r = rips_radius(N, d)
    dist = distance_matrix(pts, pts)

```

```

adj = ((dist < r) & (dist > 0)).astype(np.int8)
return pts, adj, r, dist

def count_triangles(adj):
    A = adj.astype(np.int32)
    A3 = A @ A @ A
    return int(np.trace(A3)) // 6

def count_K4(adj):
    """Compte les K4 (4-cliques) – chaque K4 compté exactement 1 fois
    via l'arête lex-min (i<j) puis paires (k,l) > j avec A[k,l]=1."""
    A = adj.astype(bool)
    N = A.shape[0]
    n_k4 = 0
    for i in range(N - 1):
        Ai = A[i]
        for j in range(i + 1, N):
            if not Ai[j]:
                continue
            common = Ai & A[j]
            common_above = np.where(common)[0]
            common_above = common_above[common_above > j]
            if len(common_above) < 2:
                continue
            sub = A[np.ix_(common_above, common_above)]
            n_k4 += int(np.triu(sub, 1).sum())
    return n_k4

def compute_S_sim(adj, n_tri=None):
    deg = adj.sum(axis=1)
    mu = deg.mean()
    if mu < 1e-9:
        return -1e9
    D = deg.std() / mu
    if n_tri is None:
        n_tri = count_triangles(adj)
    return float(np.log(1 + n_tri) - BETA * D)

def mcmc_step(adj, pool, rng, S_current):
    """Pas Metropolis edge-swap (V109)."""
    N = adj.shape[0]
    edges = np.argwhere(np.triu(adj, 1))
    if len(edges) == 0:
        return adj, S_current, False
    e_idx = rng.integers(len(edges))
    u_rm, v_rm = edges[e_idx]
    # Trouver une non-arête dans le pool

```

```

u_add = v_add = -1
for _ in range(100):
    i, j = rng.integers(N, size=2)
    if i != j and pool[i, j] == 1 and adj[i, j] == 0:
        u_add, v_add = i, j
        break
if u_add < 0:
    return adj, S_current, False
adj_new = adj.copy()
adj_new[u_rm, v_rm] = adj_new[v_rm, u_rm] = 0
adj_new[u_add, v_add] = adj_new[v_add, u_add] = 1
S_new = compute_S_sim(adj_new)
delta = (S_new - S_current) * N / TAU
if delta > 0 or rng.random() < np.exp(min(0.0, delta)):
    return adj_new, S_new, True
return adj, S_current, False

def run_mcmc(N, d, seed, n_steps, sample_every):
    pts, adj, r, dist = build_rips_dim(N, d, seed)
    pool = ((dist < 1.5 * r) & (dist > 0)).astype(np.int8)
    rng = np.random.default_rng(seed + 9991)
    S_cur = compute_S_sim(adj)
    history = {'step': [], 'n_tri': [], 'n_K4': [], 'S': []}
    for step in range(n_steps):
        adj, S_cur, _ = mcmc_step(adj, pool, rng, S_cur)
        if step % sample_every == 0 or step == n_steps - 1:
            n_tri = count_triangles(adj)
            n_k4 = count_K4(adj)
            history['step'].append(step)
            history['n_tri'].append(n_tri)
            history['n_K4'].append(n_k4)
            history['S'].append(S_cur)
    return adj, history

# =====
# TEST A
def test_A():
    print("\n" + "="*68)
    print("TEST A : Sweep dimensionnel d_amb ∈ {1,2,3,4,5} (Mermin-Wagner)")
    print("="*68)
    print(f"N={N}, n_steps={N_STEPS}, seeds={SEEDS}")
    print(f"Hypothèse : la formation K4 doit être supprimée pour d_amb<3,")
    print(f"transitoire/abondante pour d_amb≥3.")
    print()
    results = {}
    for d in DIMS:
        tri_eq, k4_eq = [], []

```

```

t0 = time.time()
for seed in SEEDS:
    _, hist = run_mcmc(N, d, seed, N_STEPS, SAMPLE_EVERY)
    # Equilibre = moyenne sur le dernier tiers
    tail = max(1, len(hist['n_tri']) // 3)
    tri_eq.append(np.mean(hist['n_tri'][-tail:]))
    k4_eq.append(np.mean(hist['n_K4'][-tail:]))
    results[d] = {
        'tri_mean': float(np.mean(tri_eq)),
        'tri_std': float(np.std(tri_eq)),
        'k4_mean': float(np.mean(k4_eq)),
        'k4_std': float(np.std(k4_eq)),
        'tri_per_seed': [float(x) for x in tri_eq],
        'k4_per_seed': [float(x) for x in k4_eq],
    }
dt = time.time() - t0
print(f"d={d}  n_tri = {results[d]['tri_mean']:7.1f} ± {results[d]['tri_std']:5.1f}
      n_K4 = {results[d]['k4_mean']:7.2f} ± {results[d]['k4_std']:5.2f}
      [{dt:.1f}s]")
print("\n--- Résumé Test A ---")
print(f"{'d_amb':<8}{'n_tri':<22}{'n_K4':<22}{'ratio K4/tri':<14}")
for d in DIMS:
    r = results[d]
    ratio = r['k4_mean'] / max(r['tri_mean'], 1e-9)
    print(f"{'d':<8}{r['tri_mean']:6.1f} ± {r['tri_std']:5.1f}
          {r['k4_mean']:6.2f} ± {r['k4_std']:5.2f} {ratio:.4f}")
return results

# =====
# TEST B
def test_B():
    print("\n" + "="*68)
    print("TEST B : Cascade n_K4 vs n_tri (d_amb=3, exposant cascade)")
    print("="*68)
    print(f"Hypothèse : si scaling ER pur, n_K4 ∝ n_tri^b avec b=2.")
    print(f"b > 2 ⇒ K4 sur-représentés (signature 'matière').")
    print(f"b < 2 ⇒ K4 supprimés.")
    print()
    all_tri, all_k4 = [], []
    for seed in SEEDS:
        _, hist = run_mcmc(N, 3, seed, N_STEPS, SAMPLE_EVERY)
        # On garde le second tiers + dernier tiers (équilibre)
        keep_from = len(hist['n_tri']) // 3
        all_tri.extend(hist['n_tri'][keep_from:])
        all_k4.extend(hist['n_K4'][keep_from:])
    tri_arr = np.array(all_tri, dtype=float)
    k4_arr = np.array(all_k4, dtype=float)

```

```

mask = (tri_arr > 0) & (k4_arr > 0)
if mask.sum() < 5:
    print(f"Pas assez de points (>0) : {mask.sum()}. Test B non-concluant.")
    return None
log_t = np.log(tri_arr[mask])
log_k = np.log(k4_arr[mask])
b, log_a = np.polyfit(log_t, log_k, 1)
pred = b * log_t + log_a
ss_res = np.sum((log_k - pred)**2)
ss_tot = np.sum((log_k - log_k.mean())**2)
r2 = 1 - ss_res / max(ss_tot, 1e-12)
n_points = int(mask.sum())
print(f"n_points utilisés : {n_points}")
print(f"Fit log(n_K4) = {b:.3f} · log(n_tri) + {log_a:.3f}")
print(f"a = {np.exp(log_a):.4f}, b = {b:.3f}")
print(f"R² = {r2:.3f}")
print(f"Δb vs ER (b=2) : {b - 2.0:+.3f}")
return {'a': float(np.exp(log_a)), 'b': float(b),
        'r2': float(r2), 'n_points': n_points,
        'delta_b_vs_ER': float(b - 2.0)}

# =====
if __name__ == "__main__":
    t_start = time.time()
    print(f"Démarrage test scout. Paramètres : N={N}, n_steps={N_STEPS}, "
          f"seeds={SEEDS}, kappa0={KAPPA0}, beta={BETA}, tau={TAU}")
    res_A = test_A()
    res_B = test_B()
    t_total = time.time() - t_start
    print(f"\n{'='*68}")
    print(f"TEMPS TOTAL : {t_total:.1f}s")
    print(f"{'='*68}")
    # Sauvegarde JSON
    out = {
        'params': {'N': N, 'n_steps': N_STEPS, 'seeds': SEEDS,
                   'kappa0': KAPPA0, 'beta': BETA, 'tau': TAU},
        'test_A': res_A,
        'test_B': res_B,
        'wall_time_s': t_total,
    }
    with open('/home/claude/scout_results.json', 'w') as f:
        json.dump(out, f, indent=2)
    print(f"Résultats sauvés dans /home/claude/scout_results.json")

```